Команда «Ультратех», задача № 9

Тренажёр диспетчера ДДС

Общее техническое описание

Учебное программное обеспечение для подготовки диспетчеров дежурно-диспетчерских служб города Москвы с использованием искусственного интеллекта.

Хакатон «Лидеры цифровой трансформации 2026»
Москва, редакция от 17.10.2026

Содержание

1. Назначение 3

2. Кто работает в системе 3

3. Функции и возможности 3

4. Как устроена система 4

5. Требования к оборудованию 6

6. Порядок установки 6

6.1. Сервер с интернетом 6

6.2. Сервер без интернета 6

6.3. Первый вход 7

6.4. Проверка после установки 7

7. Настройка 7

7.1. Где настраивается 7

7.2. Основные настройки 8

7.3. Модели ИИ 8

7.4. Телефония 8

7.5. Резервные копии 9

8. Возможные неисправности и их устранение 9

9. Ограничения 10

1. Назначение

Тренажёр диспетчера ДДС — учебная программа для подготовки диспетчеров дежурно-диспетчерских служб (ДДС) города Москвы, которые работают с карточками происшествий системы-112.

Программа повторяет рабочее место АРМ-112: журнал происшествий и карточку в том виде, в каком их видит диспетчер. Обучаемый получает учебные карточки, принимает их в работу, звонит аварийной бригаде и руководителям, принимает доклады о выполнении и закрывает карточку. Система записывает каждое действие, сама оценивает работу и подсказывает, что подтянуть.

Пример. Обучаемому приходит карточка «засор мусоропровода, ул. Профсоюзная, 13». Он принимает её за 30 секунд, звонит начальнику аварийной бригады, называет адрес и получает номер наряда, ставит статусы «Прибытие», «Проведение работ», «Работы завершены». После этого он видит балл, свои ошибки словами памятки заказчика и совет на следующее занятие.

Тренажёр не обрабатывает настоящие вызовы, не связан с реальными службами и не хранит персональные данные граждан: все происшествия учебные.

2. Кто работает в системе

Роль	Что делает
Обучаемый (диспетчер ДДС)	Работает на копии рабочего места: журнал, карточка, звонки, статусы. Видит свой разбор, прогресс и справочник. В интерфейсе роль называется «Обучающийся»
Преподаватель	Создаёт группы и занятия, выбирает сложность, наблюдает за обучаемыми во время занятия, смотрит отчёты и аналитику, ведёт справочник телефонов и учебные сценарии
Администратор	Заводит пользователей, следит за состоянием системы, журналом аудита, резервными копиями и настройками
ИИ	Пишет каждому обучаемому свои карточки под его ошибки, играет собеседников по телефону, оценивает действия и даёт рекомендации

Схема работы, как просил заказчик: **преподаватель и ИИ → обучаемый ДДС → результат → рекомендации для дальнейшего обучения**.

3. Функции и возможности

Рабочее место обучаемого. Журнал происшествий и карточка повторяют скриншоты живого АРМ-112: те же колонки, статусы и расположение полей. Обучаемый принимает карточку (статус «Принята») или отклоняет её («Не принята» с причиной), ведёт ход работ статусами с номером наряда и комментарием, отмечает ошибки в данных карточки. На первичный статус даётся норматив (по умолчанию 30 секунд), опоздание отмечается как «Не оповещено».

Звонки. В правом нижнем углу экрана — зелёный кружок виртуального телефона: клавиатура для набора и справочник номеров своей организации (до 10 номеров на службу, их задаёт преподаватель). Собеседники: руководитель организации, начальник отдела реагирования, начальник и специалист аварийной бригады. Трубки на вкладках служб в карточке звонят дежурным этих служб, а на вкладке своей службы — начальнику аварийной бригады. Заявителю можно

позвонить, только если преподаватель разрешил это в занятии. Бригада сама перезванивает с докладами о прибытии и выполнении работ.

Сложность. Три уровня:

- 1 — простое бытовое происшествие, данные карточки верные, собеседники понятливые;
- 2 — сложнее: в карточке одна ошибка оператора 112 (подвох), собеседник один раз путает данные, и диспетчер должен его поправить;
- 3 — чрезвычайная ситуация, две ошибки в карточке, собеседники торопят, на месте бывают осложнения, у службы может не оказаться машин. В обычном занятии на этой сложности карточки идут валом: около четырёх в минуту в течение пяти минут.

Задания формирует ИИ. Преподаватель выбирает группу и сложность и нажимает «Начать». Дальше ИИ сам пишет каждому обучаемому свои карточки, учитывая его прошлые ошибки, выдаёт их, оценивает работу и даёт рекомендации. Карточки приходят по одной: следующая пишется после оценки предыдущей, на любой сложности, без вала. Преподаватель в это время только наблюдает.

Оценка и разбор. Балл складывается из понятных частей: решение по карточке, время против норматива, цепочка статусов, звонки, комментарии и номер наряда, найденные ошибки в карточке, грамотность. Каждая ошибка названа словами памятки заказчика «Работа на АРМ-112» со ссылкой на страницу. После вала карточек отдельно показывается сводка: сколько пришло, сколько обработано, сколько с опозданием, взята ли первой чрезвычайная ситуация.

Кабинет преподавателя. Группы, занятия, наблюдение за каждым обучаемым во время занятия, отчёт с выгрузкой в PDF, Excel и CSV, исправление оценки с указанием причины, аналитика группы, прогноз готовности к аттестации, библиотека учебных сценариев, методические материалы.

Кабинет администратора. Пользователи и роли, службы, состояние системы, журнал аудита с проверкой целостности, резервные копии, настройки.

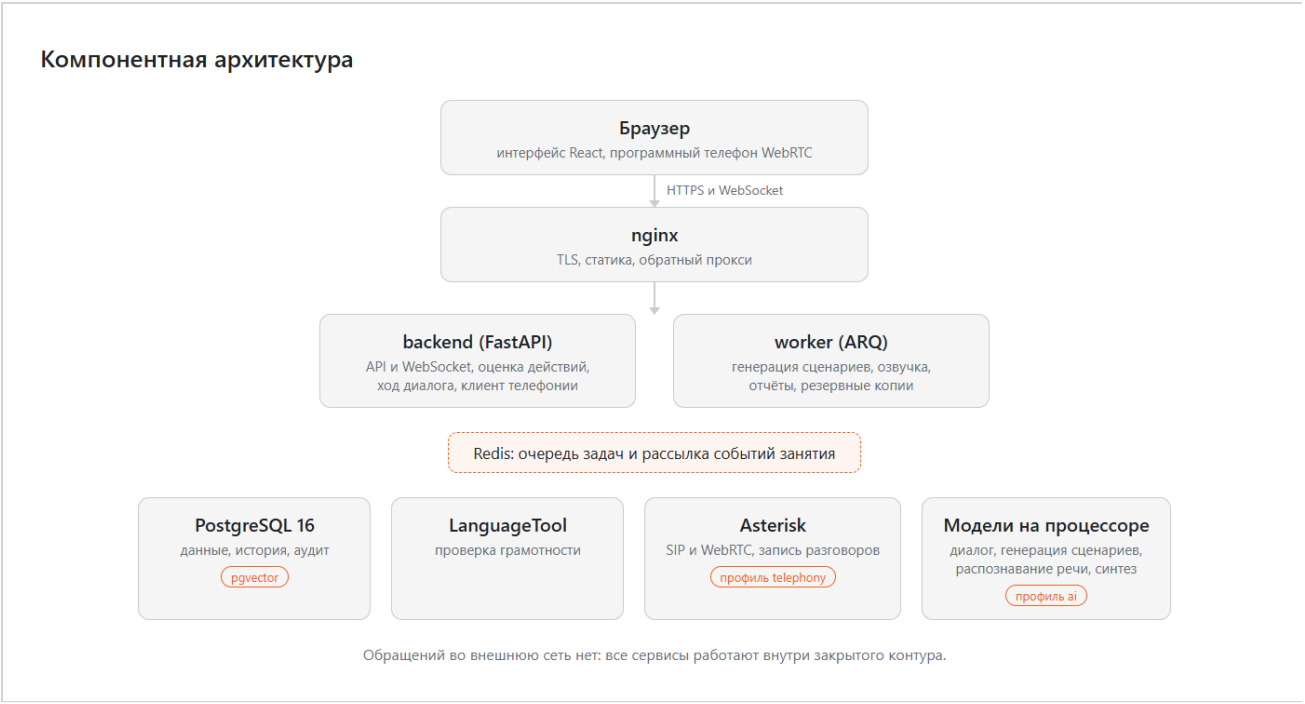
Справочник. Памятка «Работа на АРМ-112», методические материалы преподавателя и классификатор происшествий с поиском по словам.

Данные заказчика. Классификатор происшествий (1283 типа, по нему подбираются службы), памятка по работе с АРМ-112 (статусы и типичные ошибки), 96 учебных ситуаций из билетов, скриншоты живой системы.

Дополнительно в программе есть режим оператора 112 (приём звонка заявителя и заполнение карточки). По рекомендации заказчика он скрыт и включается администратором отдельным пользователям.

4. Как устроена система

Система ставится на один сервер учебного класса. Рабочие места преподавателя и обучаемых — это обычные компьютеры с браузером в той же сети, ставить на них ничего не нужно.



Состав системы

Все части программы запускаются в контейнерах Docker одной командой:

Часть	Что делает
Веб-сервер (nginx)	Отдаёт интерфейс браузеру по защищённому соединению
Сервер приложения (backend)	Занятия, карточки, оценка, разговоры, телефония
Фоновые задачи (worker)	Написание карточек, озвучка, отчёты
База данных (PostgreSQL)	Пользователи, занятия, результаты, журнал аудита
Очередь (Redis)	Фоновые задачи и события занятия
Проверка грамотности (LanguageTool)	Ошибки в комментариях обучаемого
Резервные копии (backup)	Ежедневная копия базы
Модели ИИ (llm-dialog, llm-gen)	Собеседники по телефону и написание карточек
Распознавание речи (stt)	Переводит голос обучаемого в текст
Телефония (Asterisk)	Звонки из браузера и с настольного IP-телефона

Работа без интернета. У заказчика все модели ИИ, распознавание и синтез речи работают на самом сервере, без интернета и без видеокарты, только на процессоре. На демонстрационном стенде для скорости написание карточек отдано облачной модели — это одна настройка (раздел 7.3), код при этом тот же. Если модель не ответила, карточка собирается по шаблону, и занятие не останавливается.

5. Требования к оборудованию

Сервер:

- процессор от 6 ядер (Intel Core i7, Xeon или аналог), видеокарта не нужна;
- оперативная память от 16 ГБ, рекомендуется 32 ГБ;
- диск SSD, свободно от 100 ГБ (программа с моделями занимает около 40 ГБ);
- операционная система Linux (Ubuntu 22.04 или 24.04);
- Docker с docker compose (версия 2).

Демонстрационный стенд работает на 6 ядрах и 12 ГБ памяти с облачной моделью для карточек.

Рабочие места: браузер Chrome, Яндекс.Браузер или Firefox, гарнитура с микрофоном. Можно подключить настольный IP-телефон.

Сеть: рабочие места должны видеть сервер по адресу HTTPS (порт 443). Для настольных IP-телефонов дополнительно открываются порт SIP (5060) и диапазон голосовых портов (10000–10100).

Нагрузка (замерено): до 100 пользователей в кабинетах одновременно с откликом не больше 2 секунд, 20 обучаемых одновременно ведут карточки. Голосовые разговоры с моделями на процессоре тяжелее: на 6 ядрах рассчитывайте на 3–4 разговора одновременно.

6. Порядок установки

Есть два варианта: сервер с интернетом и сервер без интернета (закрытый контур). Команды вводятся в терминале сервера.

6.1. Сервер с интернетом

```
git clone https://github.com/Vadik145-beep/lct-112-simulator.git
cd lct-112-simulator
./start.sh
```

Скрипт сам скачает модели ИИ (около 9 ГБ), создаст файл настроек `.env`, соберёт и запустит систему. Первый запуск занимает от 30 минут до часа. В конце скрипт напечатает адрес, логины и пароль.

6.2. Сервер без интернета

Поставка готовится на любом компьютере с интернетом (Linux, Docker, Git, около 25 ГБ свободного места):

```
git clone https://github.com/Vadik145-beep/lct-112-simulator.git
cd lct-112-simulator
./scripts/fetch_models.sh
./scripts/offline_bundle.sh --with-models
```

Готовая папка `offline-bundle` (около 11 ГБ) переносится на сервер флешкой, диском или по локальной сети. На сервере нужен только Docker:

```
cd offline-bundle
sudo ./install.sh
```

Установщик проверит контрольные суммы файлов, загрузит готовые образы, установит систему в `/opt/lct/app` и запустит её. Во время установки сервер ничего не скачивает из интернета.

6.3. Первый вход

Откройте в браузере адрес, который напечатал скрипт, например `https://192.168.1.50`. Браузер предупредит о сертификате: у системы свой центр сертификации. Нажмите «Дополнительные» → «Перейти на сайт». Чтобы предупреждения не было, установите на рабочие места корневой сертификат:

```
docker compose cp nginx:/etc/nginx/certs/ca.crt .
```

После установки созданы три пользователя:

Роль	Логин	Пароль
Администратор	`admin`	`12345678`, при первом входе система попросит сменить
Преподаватель	`teacher`	`12345678`
Обучаемый	`student`	`12345678`

Пароль по умолчанию задаётся настройкой `SEED_PASSWORD` до первого запуска.

6.4. Проверка после установки

1. Откройте `https://<адрес>/health` — должно быть `"status":"ok"`.
2. Войдите администратором, откройте «Состояние»: все плитки сервисов «работает».
3. Войдите преподавателем, создайте группу, добавьте в неё `student` и создайте занятие.
4. Войдите обучаемым: откроется журнал с первой карточкой.

7. Настройка

7.1. Где настраивается

Почти всё задаётся в файле `.env` в папке установки (образец с пояснениями — `.env.example`). После правки файла систему перезапускают:

```
docker compose up -d
```

Часть настроек администратор меняет прямо в интерфейсе, в разделе «Настройки» (руководство администратора): телефония, уровень журнала, расписание копий, порог «Не завершено».

7.2. Основные настройки

Настройка	Что задаёт
`SECRET_KEY`	Секрет для входа в систему. Обязательно поменять: `openssl rand -hex 32`
`POSTGRES_PASSWORD`, `APP_DB_PASSWORD`	Пароли базы данных. Поменять до первого запуска
`SEED_PASSWORD`	Пароль пользователей, создаваемых при первом запуске
`APP_ENV`	`production` для рабочей установки
`TLS_HOSTS`	Адреса и имена сервера для сертификата, через запятую
`HTTP_PORT`, `HTTPS_PORT`	Порты веб-интерфейса, по умолчанию 80 и 443
`SHOW_DEMO_CREDENTIALS`	Таблица логинов и паролей на странице входа. Только для демонстрации, у заказчика `false`
`QUICK_LOGIN`	Кнопки «Войти как …» без пароля. Только для демонстрации, у заказчика `false`
`TZ`	Часовой пояс, по умолчанию `Europe/Moscow`
`LOGIN_MAX_ATTEMPTS`, `LOGIN_LOCK_MINUTES`	Блокировка входа после неудачных попыток: 5 попыток, 15 минут

7.3. Модели ИИ

Настройка	Что задаёт
`LLM_DIALOG_MODEL`	Модель собеседников. На стенде Qwen2.5-3B
`LLM_GEN_URL`	Адрес модели, которая пишет карточки. По умолчанию локальная
`STT_MODEL`	Распознавание речи: `faster-whisper-base` быстрее, `faster-whisper-small` точнее
`TTS_PROVIDER`	Голос собеседников: `auto` — Silero, при его отсутствии Piper
`LLM_THREADS`	Сколько ядер процессора отдать модели

У заказчика все адреса моделей локальные, и система работает без интернета. Облачная модель подключается только явно: нужно поставить `ALLOW_EXTERNAL_AI=true` и указать `LLM_GEN_URL`, `LLM_GEN_API_KEY` и `LLM_GEN_API_MODEL`. Без разрешения `ALLOW_EXTERNAL_AI` система с внешним адресом модели не запустится и сообщит причину. Так сделано на демонстрационном стенде.

7.4. Телефония

Звонки из браузера работают без отдельной настройки. Для настольных IP-телефонов:

Настройка	Что задаёт
`TELEPHONY_ENABLED`	`true` — включить телефонию

Настройка	Что задаёт
`TELEPHONY_EXTERNAL_IP`	Адрес сервера, который видят телефоны
`SIP_PORT`	Порт SIP, по умолчанию 5060
`TELEPHONY_RTP_START` , `TELEPHONY_RTP_END`	Диапазон голосовых портов
`ARI_PASSWORD`	Пароль управления телефонией. Поменять

Запуск телефонии: `docker compose --profile telephony up -d` . В настольном телефоне указываются адрес сервера, логин и пароль SIP из кабинета администратора (кнопка «SIP-учётка» у обучаемого), кодеки G.722 и PCMA.

7.5. Резервные копии

`BACKUP_TIME` — время ежедневной копии (по умолчанию 03:00), `BACKUP_KEEP` — сколько копий хранить (по умолчанию 14). То же меняется в интерфейсе администратора.

8. Возможные неисправности и их устранение

Что видно	Причина	Что сделать
Страница не открывается сразу после запуска	Модели ещё загружаются	Подождать 2–3 минуты и обновить страницу
Браузер пишет «Подключение не защищено»	Свой сертификат системы	«Дополнительные» → «Перейти на сайт» или установить корневой сертификат (раздел 6.3)
При установке `port is already allocated`	Порт 80 или 443 занят другой программой	В `.env` поставить `HTTP_PORT=8080` , `HTTPS_PORT=8443` , запустить снова, открывать адрес с `:8443`
`docker: command not found` или `permission denied`	Нет Docker или прав	Установить Docker, запускать через `sudo`
Всё работает медленно, сервер зависает	Мало памяти	Нужно от 16 ГБ. Временно можно уменьшить `LLM_THREADS` или перейти на `STT_MODEL=faster-whisper-base`
Пользователь не может войти, система пишет о неудачных попытках	5 неверных паролей подряд, вход закрыт на 15 минут	Подождать 15 минут; если пароль забыт — администратор сбрасывает его
Собеседник по телефону не слышит обучаемого	Браузеру не дан доступ к микрофону	Разрешить микрофон в адресной строке браузера, проверить гарнитуру
Собеседник отвечает невпопад	Распознавание ошибается на шуме	Говорить чётче, ближе к микрофону; можно перейти на `STT_MODEL=faster-whisper-small`
Карточки ИИ однотипные или приходят с задержкой	Модель генерации не отвечает, карточки собираются по шаблону	В «Состоянии» проверить плитку «Модель генерации»; при «не отвечает» перезапустить: `docker compose --profile ai restart`

Что видно	Причина	Что сделать
В оценке «грамотность не проверена»	Не отвечает LanguageTool	`docker compose restart languagetool`; остальная оценка работает
Настольный телефон не звонит	Телефония выключена или закрыты порты	Проверить `TELEPHONY_ENABLED=true`, профиль `telephony`, порт SIP и голосовые порты на межсетевом экране
В «Копиях» надпись «служба backup не отмечалась»	Остановлен контейнер копий	`docker compose up -d backup`
В «Состоянии» плитка «не отвечает»	Остановлен один из сервисов	`docker compose ps` покажет, какой; `docker compose up -d` поднимет его

Общая проверка: `docker compose ps` — все сервисы должны быть в состоянии `running` или `healthy`. Журнал сервиса: `docker compose logs --tail 100 backend`.

9. Ограничения

- Тренажёр учебный: не принимает настоящие вызовы и не передаёт данные в реальные службы.
- Собеседник на процессоре отвечает через 1,5–3 секунды после реплики обучаемого. Собеседник по телефону говорит утверждёнными репликами, поэтому на неожиданный вопрос может ответить «не понял, повторите».
- Распознавание речи ошибается на номерах домов и именах при плохом микрофоне. Преподаватель видит расшифровку разговора и может исправить оценку с указанием причины.
- Резервная копия содержит базу данных. Записи разговоров и озвученные реплики лежат отдельно (том `storage`), их копируют вместе с сервером.
- Голосов на процессоре одновременно немного (раздел 5). Для большого класса модели выносят на отдельный сервер: их адреса задаются настройками.